RuralCare Route

The Doctor, Ambulance & Medicine Routing Problem — Final MVP Blueprint

CodeRush 1.0 — AlgoWebathon Submission

August 2026

1. Executive Summary

RuralCare Route is an algorithm-first web application that solves rural emergency healthcare coordination. A decentralized rural network connects hundreds of villages to a handful of hospitals with scarce specialists, limited ambulances, and volatile medicine stock. When an emergency happens, the system must decide — in real time — **who treats the patient, who drives them there, and what road they take**, while constantly reacting to beds filling up, specialists going off-duty, medicine running out, and roads closing.

The engine is powered by **Dijkstra / A*** shortest-path search over a weighted road graph, a **binary-heap priority queue** for triage and pathfinding, and a **constraint filter + weighted cost function** for facility and ambulance selection. The UI exists to make that algorithm *visible and provable* to a judge, not to decorate it.

USP: *"We don't route patients to the nearest hospital — we route them to the nearest hospital actually capable of treating them, and we keep re-routing them the moment that stops being true."*

2. Problem Statement (Official Brief)

> A decentralized rural healthcare network connects hundreds of remote villages with scarce specialized doctors, limited ambulance transport fleets, and volatile medicine stocks. During peak influx periods, patient requests, urgency tiers, doctor shifts, and pharmacy supplies fluctuate dynamically.
>
> **Goal:** Design and build an intelligent routing and scheduling engine that dynamically dispatches ambulances, routes patients to qualified medical facilities, and manages medicine supplies while minimizing total operational cost (Travel Time + Wait Time) and respecting emergency urgency.

Benchmark parameters quoted in the brief (50,000+ nodes, 200,000+ edges, 5,000+ villages) describe the **target**

production scale, not the demo dataset — see §5.

3. What Makes This Submission Stand Out

Differentiator	Why it wins marks
Doctor-in-the-loop intake	Ambulance reaches patient → attending doctor enters emergency type, urgency, required specialist & medicine. More realistic than a patient self-diagnosing, and it cleanly separates *triage* from *routing*.
Hard-constraint filter before cost optimization	Hospitals lacking the specialist, a bed, or the medicine are **rejected outright** — never merely down-ranked. Prevents the classic bug of "nearest hospital wins even though it can't treat the patient."
Live dynamic reallocation	Mid-transit, the selected hospital's cardiologist can go off-duty or its beds can fill. The engine detects this, releases the reservation, and re-routes the ambulance to the next-best hospital — demonstrated live, not scripted.
Resource reservation, not just decrement	Beds and medicine are RESERVED the instant a hospital is chosen (not just subtracted), so two simultaneous critical requests can never double-book the same bed.
A* + Dijkstra side-by-side	Dijkstra runs as a correctness oracle; A* is the production path-finder. Showing both proves the A* heuristic is admissible — this is a direct, visible answer to "algorithmic correctness."
Decision log generated from real algorithm output	Every rejection and selection is printed with its actual reason and cost — not a hard-coded string — which is explicitly what separates high and low scores on this criterion.
Edge-case control panel	Judges can manually trigger road closures, bed-fulls, medicine depletion, and ambulance shortages on demand instead of waiting for random events.

4. Tech Stack

The brief's own submission page states a **6-hour build window** and a **client-side, in-memory simulation** — so the MVP is deliberately **frontend-only**. No backend, no DB, no auth. This maximizes time spent on the algorithm (worth the most points) instead of infrastructure.

Layer	Technology	Why
Framework	**React + Vite**	Fastest scaffold-to-running time
Language	**TypeScript**	Typed graph/queue/entity models catch bugs fast under time pressure
Styling	**Tailwind CSS**	Utility-first, rapid UI polish, no context-switching to CSS files
State	**React `useReducer` + Context**	Single authoritative in-memory state store; every dispatch action is one reducer case — doubles as an audit trail
Map	**Leaflet + React-Leaflet**	Lightweight, free, easy custom markers/overlays, no API key friction
Graph & Algorithms	**Hand-rolled TypeScript** (`Graph.ts`,	Judges are scoring *your*

	`AStar.ts`, `Dijkstra.ts`, `BinaryHeap.ts`)	implementation, not a library's
Animation	`setTimeout` step interpolation along the route polyline	"Good enough" motion without an animation library
Testing	**Vitest**	Fast unit tests for the correctness suite (§10)
Hosting	**Vercel**	One-click deploy from GitHub, matches the submission checklist

Optional, if time remains: Socket.IO + a tiny Node/Express layer purely to let two judge browsers watch the same simulation — **not required** for the core score.

Explicitly out of scope for the 6-hour build (mention only in README "Future Scope"): a real backend/database, authentication, external map/routing APIs, mobile-responsive polish, and the full 50,000-node dataset actually loaded into the browser.

5. Scope Definition

In-Scope (We WILL build)	Out-of-Scope (We will NOT build)
Graph engine: 20–100 nodes (villages + hospitals), weighted adjacency list	Massive scale in-demo (50k nodes) — designed for it, not loaded with it
Dijkstra + A* with a custom binary heap	Backend / database — all state is in-memory on the client
Doctor-entered triage → priority queue (Critical/High/Medium/Low)	Authentication / user logins
Hard-constraint facility filter (specialist, bed, medicine, reachability)	Real clinical diagnosis logic
Weighted cost function (travel + wait, α/β tunable)	Nationwide live hospital integration / government APIs
Bed & medicine **reservation** (not just decrement)	IoT hardware, drone delivery, EHR systems
Ambulance state machine (AVAILABLE → ASSIGNED → EN_ROUTE → ARRIVED)	Advanced ML / predictive models
Dynamic events: road closures, specialist going off-duty, bed fills, medicine depletion	External live traffic / real map routing APIs
Live re-routing / reallocation when the chosen facility becomes infeasible	Mobile-responsive design (desktop-first is fine)
Interactive map, live telemetry, decision log, simulation control buttons	Perfect animation — step interpolation is sufficient

6. System Architecture

```
UI Layer (React)
  ├── Interactive Map (React-Leaflet)
  ├── Live Telemetry Panel
  ├── Decision Log Panel
  └── Simulation Controls
        │
        ▼
State Store (useReducer + Context)  ──► single source of truth
        │
        ▼
Dispatch Engine
  ├── Patient Request Manager        (validates + timestamps)
  ├── Emergency Priority Queue       (binary heap, urgency-ranked)
  ├── Facility Filter                (hard constraints: specialist/bed/medicine/reachable)
```

```
   ├── Cost Function / Allocation     (α·travel + β·wait → best hospital)
   ├── Ambulance Allocator            (nearest AVAILABLE unit)
   ├── Resource Manager               (bed & medicine reservation)
   └── Routing Engine                 (A* primary, Dijkstra reference)
         │
         ▼
Dynamic Event / Simulation Engine
  (road closures, doctor shift changes, bed fills, medicine depletion, new requests)
```

7. Data Model

Entity	Key Fields
Graph Node	`id`, `lat`, `lng`, `type` (village / hospital)
Road Edge	`from`, `to`, `travelTime`, `distance`, `status` (open/closed)
Village	`id`, `nodeId`, `name`
Hospital	`id`, `nodeId`, `specialists[]`, `bedsTotal`, `bedsAvailable`, `bedsReserved`, `medicines{}`, `status`
Doctor	`id`, `specialty`, `facilityId`, `shiftStatus`
Ambulance	`id`, `nodeId`, `status`, `currentRequestId`
Medicine	`id`, `facilityId`, `quantityAvailable`, `quantityReserved`
Patient Request	`id`, `patientId`, `originNode`, `emergencyType`, `urgency`, `specialtyRequired`, `medicineRequired`, `createdAt`, `status`

8. Core MVP Workflow

```
Patient Emergency
   ↓
Ambulance dispatched to pick-up point
   ↓
Doctor/paramedic assesses patient on-site and enters:
   emergency type · urgency · required specialist · required medicine
   ↓
Request enters the Priority Queue  (CRITICAL > HIGH > MEDIUM > LOW)
   ↓
Facility Filter: reject hospitals missing specialist / bed / medicine / reachability
   ↓
Cost Function scores remaining feasible hospitals  (α·travel + β·wait)
   ↓
Best hospital selected  →  bed + medicine RESERVED
   ↓
Nearest AVAILABLE ambulance assigned  →  A* route computed
   ↓
Ambulance dispatched — Decision Log records every rejection & the winning reason
   ↓
Engine keeps monitoring hospital/road state while ambulance is en route
   ↓
   Condition changes?  ──Yes──► Release reservation → Re-filter → Re-score →
   │                              Select new hospital → Reserve → Re-route → Continue
   No
   ↓
Ambulance arrives → RESERVED becomes OCCUPIED / CONSUMED
```

9. Algorithmic Core

9.1 Graph representation — adjacency list, `Map<nodeId, Edge[]>`, O(V + E) space. Each edge carries `travelTime`, `distance`, and `status`.

9.2 Pathfinding — Dijkstra is the correctness baseline; **A*** (`f(n) = g(n) + h(n)`, `h(n)` = Euclidean distance to the destination) is the primary router. On identical static graphs both must return the same optimal cost — this is the live proof of correctness for judges. Complexity: **O((V + E) log V)** with a binary heap.

9.3 Emergency priority queue — binary min-heap keyed as `(urgencyRank, waitingTime)`; lower key = served first.

9.4 Facility selection — hard constraints, then cost:

1. Specialist available? 2. Bed available? 3. Medicine available (if required)?
2. Reachable under current road state?

Only facilities passing all four are scored:

```
Score(Hospital) = α · (Travel Time / MaxTravel) + β · (Wait Time / MaxWait)
        default: α = 0.6, β = 0.4   (tunable live for the demo)
```

9.5 Ambulance allocation — among AVAILABLE units, pick the minimum-ETA ambulance to the patient's origin node; if none is free, the request waits in the priority queue.

9.6 Reservation, not decrement — selecting a hospital immediately moves one bed / medicine unit from *available* to *reserved*, preventing a second simultaneous critical request from claiming the same resource. On arrival: `RESERVED → OCCUPIED/CONSUMED`. On cancel/reroute: `RESERVED → RELEASED`.

10. Dynamic Events & Correctness Testing

Event	Engine Response
Road closes	Edge marked inactive → invalidate affected route → re-run A*
Specialist goes off-duty	Facility eligibility updated instantly; reservation released & re-filtered if it was already chosen
Bed fills / medicine hits zero	Facility removed from candidates immediately
Ambulance frees up	Priority queue re-evaluated for the next waiting request
Simultaneous critical requests	Priority queue + atomic resource reservation → no double-booking

Correctness test checklist (unit tests, not just the demo): - A* cost == Dijkstra cost on identical static graphs - Closed edges never appear in a returned route - Facilities missing a hard constraint are never selected - One ambulance / bed / medicine unit is never allocated twice - Critical requests always outrank Medium/Low in queue order - Determinism: identical input state → identical decision

11. Edge-Case Resilience Matrix (Demo Buttons)

Edge Case	Trigger (UI Button)	Expected Behavior
No direct route	"Block Road"	A* finds an alternate path or logs "Unreachable" — never crashes
Nearest specialist unavailable	Request a rare specialty	Engine skips nearer, ineligible hospitals; picks nearest **eligible** one
All ambulances occupied	"Occupy All Ambulances"	Request enters the priority queue; telemetry queue count rises
Hospital beds full	"Fill Beds"	Hospital excluded; next-best hospital selected
Medicine depleted	"Deplete Medicine"	Facility excluded if medicine is mandatory for the case

Mid-transit disruption	Toggle specialist off / fill bed after dispatch	Reservation released, engine re-filters, re-scores, re-routes live

12. UI / UX Deliverables

Panel	Contents
Interactive Map	Villages, hospitals, roads, live ambulance position, animated selected route, blocked-road overlay
Live Telemetry	Ambulances (available/busy), per-hospital beds & medicine, active request counts by urgency
Decision Log	Timestamped, algorithm-generated trail: which hospitals were rejected and why, which was selected and why, bed/ambulance assigned, route cost
Simulation Controls	Generate Emergency · Block Road · Fill Beds · Deplete Medicine · Doctor Unavailable · Occupy/Free Ambulance · Reset
Benchmark Panel *(nice-to-have)*	A* vs Dijkstra timing, graph size, requests/sec

13. Judge Demo Script

1. Load the demo network (villages + hospitals) on the map.
2. Trigger a **CRITICAL cardiac emergency** at Village A.
3. Doctor enters: Cardiac Emergency, CRITICAL, Cardiologist, Cardiac Drug X.
4. Decision Log shows: *Hospital B rejected — no cardiologist (10 km). Hospital C selected — cardiologist ✓, bed ✓, medicine ✓ (25 km, but lowest feasible cost).*
5. Bed reserved, nearest ambulance assigned, A* route animates on the map.
6. **Live twist:** click “Doctor Unavailable” on Hospital C mid-transit.
7. Log shows: *Hospital C invalid — cardiologist now unavailable → reservation released → re-filtering → Hospital D selected → new bed reserved → ambulance re-routed.*
8. Trigger two simultaneous critical requests to show queue + no double-booked resources.
9. Click “Block Road” on the active route to show live A* re-routing.
10. Close with the Decision Log as the audit trail and (if built) the A* vs Dijkstra benchmark numbers.

14. 6-Hour Sprint Schedule

Time	Task	Checkpoint
0–30 min	Scaffold (`npm create vite`, Tailwind, Leaflet); hard-code 20–100 sample nodes/edges	Project runs, mock data loads
30–90 min	Implement `BinaryHeap`, `dijkstra()`, `aStar()`; console-test	Pathfinding verified correct
90–150 min	Build the dispatch engine: filter → score → reserve → assign ambulance	End-to-end console dispatch works
150–210 min	Map integration: render nodes/edges/ambulance, “Generate Emergency” flow	Clicking a village triggers a full dispatch on the map
210–270 min	Telemetry + Decision Log wired to real engine state	UI updates live from actual decisions

270–330 min	Edge-case buttons + dynamic reallocation + route animation	Full resilience demo works live
330–360 min	Polish, write `README.md`, push to GitHub, deploy to Vercel	Submission ready

15. Submission Checklist

- ☐ **GitHub repo** with clean, incremental commit history (commit every ~30 min)
- ☐ **README.md**: problem statement, algorithm design + complexity, setup instructions, live deployment link
- ☐ **Live deployment** on Vercel that actually works
- ☐ *(Bonus)* Short screen-recorded demo walkthrough linked in the README
- ☐ README explicitly states the O((V + E) log V) complexity and how the design scales toward the 50,000-node / 200,000-edge target (in-memory → indexed DB, single-process → distributed queue) — proves you understood the target scale without needing to load it in a 6-hour demo

16. Success Criteria

The MVP is complete when a judge can: generate an emergency → watch the doctor intake → see infeasible hospitals rejected with a real reason → see the best hospital selected and its bed/medicine reserved → see the nearest ambulance dispatched along an animated A* route → break the plan (road closure, specialist going off-duty, full beds, depleted medicine, ambulance shortage, simultaneous criticals) → and watch the engine **detect it and re-route live**, with every step explained in the Decision Log.

17. Future Scope (mention briefly, don’t build)

Real hospital data via secure APIs · offline-first mobile app · local-language voice interface · live ambulance GPS · predictive ambulance positioning · medicine stock prediction · government command-center dashboard · full 50k-node production deployment with a persistent database and distributed queue.